



Rapport de Projet : Codage de Huffman

Auteurs : Raphaël LANDRY, Mathis MAGRON
Encadrés par : Singh NEERAJ

Année universitaire 2024/2025

Table des matières

1	Résumé	2
2	Introduction	2
3	Architecture des modules	2
4	Présentation des principaux choix effectués	6
5	L'algorithme Compresser	6
6	L'algorithme Décompresser	7
7	Tests de nos programmes	8
8	Difficultés rencontrées et solutions	9
9	Organisation de l'équipe	9
10	Bilan technique	10
11	Bilans personnels	10

1 Résumé

Nous nous intéressons dans ce rapport à l'implémentation du codage de Huffman en langage ADA afin d'écrire deux programmes, le premier qui compresse des fichiers en utilisant le codage Huffman et le second qui les décompresse. Vous trouverez dans ce rapport une introduction au problème traité, nos choix d'architecture des modules et types utilisés, une présentation des principaux choix réalisés. Mais aussi la présentation des algorithmes `compresser.adb` et `decompresser.adb` et les différents tests effectués. Puis en fin de rapport nous expliquons les difficultés rencontrées et les solutions qui en découlent ainsi qu'un bilan technique et un bilan personnel du projet.

2 Introduction

La compression de fichiers est un sujet très important en programmation. En effet, que ce soit des images (avec le format JPEG) ou des sons (avec le format MP3) la compression est omniprésente. Elle permet notamment de réduire l'espace de stockage utilisé par un fichier ou de transmettre plus rapidement des données.

Dans ce projet, nous nous sommes intéressés à l'un de ces algorithmes de compression : le codage de Huffman que nous traiterons uniquement pour des fichiers textes. Ce codage, apparu en 1952, fonctionne de la manière suivante ; on définit pour chaque caractères présent dans le fichier un nouveau codage dont la taille varie selon leur fréquence d'apparition dans le fichier texte. Les caractères dont la fréquence est élevée seront codés sur moins de bits que ceux dont la fréquence est faible.

L'algorithme de Huffman permet de définir un code optimal au sens de la plus courte longueur pour un codage par symbole.

Le but de ce projet est d'implémenter un algorithme en ADA qui utilise le codage de Huffman pour compresser un fichier texte (en format ASCII) , ainsi qu'un algorithme qui permet de décompresser le fichier et donc de retrouver le fichier originel.

3 Architecture des modules

Pour l'implémentation de ce projet, nous avons créé un package `LCA_ABR` qui contient les différents types utilisés, à savoir :

- `T_Octet` : modélise un Octet en ADA.
- `Tableau_Frequence` : tableau de 257 entiers qui permet le stockage de la fréquence du caractère à l'indice égale à l'entier associé à son code ASCII.
- `T_Liste_Symboles` : pointeur vers `T_Cellule_Symboles`.

- T_Cellule_Symboles : un enregistrement qui contient
 - Symbole : Le caractère en Unbounded_String.
 - Code : String de taille 8 qui est le code binaire ASCII.
 - Suivant : Pointeur de type T_Liste_Symboles vers la cellule suivante.
- T_Table_Encode : pointeur vers T_Cellule_Encodage qui est la table d'encodage sous la forme d'une liste chaînée associative.
- T_Cellule_Encodage : un enregistrement qui contient
 - Char : Le caractère associé en Unbounded_String.
 - Code : Son code binaire en Unbounded_String.
 - Suivant : Pointeur de type T_Table_Encodage vers la cellule suivante.
- T_Data : un enregistrement qui contient les données dans l'arbre binaire.
 - Frequence : la fréquence en Entier.
 - Character : le caractère en Unbounded_String.
- T_ABR : pointeur vers T_Node, modélise l'arbre d'Huffman.
- T_Node : un enregistrement qui modélise un noeud de l'arbre.
 - Data : de type T_Data défini plus tôt.
 - left : sous arbre gauche de type T_ABR.
 - right : sous arbre droit de type T_ABR.
- T_LCA_ABR : Pointeur vers T_Cellule, c'est une liste chaînée associative d'arbres binaires.
- T_Cellule : un enregistrement qui contient
 - Abr : arbre de type T_ABR;
 - Suivant : Pointeur de type T_LCA_ABR vers la cellule suivante.

Voici ci-dessous la spécification des procédures présentes dans le module.

```
-- Initialisation de l'arbre binaire Abr a null
procedure Init (Abr : out T_ABR);

-- Retourne True si l'arbre est vide, sinon retourne
  False
function Abr_Vide (Abr : in T_ABR) return Boolean;

-- Associe deux arbres comme étant les branches gauche
  et droite d'un autre arbre
procedure Associer (Abr : in out T_ABR; Abr_G, Abr_D : in
  T_ABR);
```

```

-- Initialise une liste chaîné associative d'arbres
  binaires
procedure Initialiser_LCA_ABR(L_Abr: out T_LCA_ABR);

-- Retourne True si la LCA d'arbres est vide, sinon
  retourne False
function Est_Vide (L_Abr : in T_LCA_ABR) return Boolean;

-- Retourne la taille d'une LCA d'arbres
function Taille (L_Abr : in T_LCA_ABR) return integer;

-- Enregistre un arbre dans une LCA d'arbres de manière
  croissante en fonction de la fréquence a la base de l'
  'arbre d'entrée
procedure Enregistrer_Croissant (L_Abr : in out
  T_LCA_ABR ; Abr : in T_ABR);

-- Supprime un arbre d'une LCA d'arbres
procedure Supprimer (L_Abr : in out T_LCA_ABR ; Abr : in
  T_ABR);

-- Affichage d'une LCA d'arbre pour des tests
procedure Afficher_Liste_Arbre (L_Abr : in T_LCA_ABR);

-- Construit l'arbre de Huffman (Arbre_Huffman) a partir
  de la LCA d'arbre initiale issue de la première
  lecture du fichier
procedure Huffman (L_Abr : in out T_LCA_ABR;
  Arbre_Huffman : out T_ABR);

-- Affiche l'arbre de Huffman
procedure Afficher_Bavard_Huffman (Arbre :in T_ABR;
  Indentation : in out Unbounded_String);

-- Version Mathis
procedure Afficher_Bavard_Huffman2 (Arbre :in out T_ABR;
  Indentation : in out Unbounded_String);

-- Construit le code Infixe (Code_Infixe) en lisant l'
  arbre de Huffman (Abr)
procedure Infixe (Abr : in T_ABR; Code_Infixe : in out
  Unbounded_String);

-- Construit la tableau de fréquence initiale en lisant
  le fichier
procedure Creer_Tableau_Frequence_Symbole(

```

```

        Tableau_Frequence_Symbole : in out Tableau_Frequence;
        Nom_Fichier_txt : in Unbounded_String);

-- Ajoute une cellule au début de la LCA de symboles
Liste_Symboles
procedure Ajouter_Cellule_Debut(Liste_Symboles : in out
    T_Liste_Symboles; Symbole : Unbounded_String; Code :
    String);

-- Ajoute une cellule a la fin de la LCA de symboles
Liste_Symboles
procedure Ajouter_Cellule_Fin(Liste_Symboles : in out
    T_Liste_Symboles; Symbole : Unbounded_String; Code :
    String);

-- Creer la LCA de symboles issue de la table d'encodage
(=> arbre de huffman)
procedure Generer_Liste_Symboles(Table_Encodage : in
    T_Table_Encodage; Liste_Symboles : in out
    T_Liste_Symboles );

-- Affichage de la liste de symboles pour des tests
procedure Afficher_Liste_Symboles(Liste_Symboles : in
    T_Liste_Symboles);

-- Genere la table d'encodage (Table) issue de l'arbre
de Huffman (Arbre_Huffman)
procedure Generer_Table_Encodage(Arbre_Huffman : in
    T_ABR; Table : in out T_Table_Encodage);

-- Retourne l'encodage de Huffman du Character donné en
entrée
function Trouver_Codage(Liste : T_Table_Encodage;
    Character : Unbounded_String) return Unbounded_String;

-- Affichage de la table d'encodage pour des tests
procedure Afficher_Table_Encodage(Table : in
    T_Table_Encodage);

-- Encodage du texte a compresser via la table d'
encodage
procedure Codage_Texte_Huffman(Code_Texte_Huffman : out
    Unbounded_String; Nom_Fichier_txt : in String;
    Table_Encodage : in T_Table_Encodage);

-- Transforme une liste de symboles en Unbounded String

```

```

procedure String_Liste_Symbole(Liste_Symboles : in
    T_Liste_Symboles; Liste_Fichier_Symboles : out
    Unbounded_String);

-- Complète un Unbounded_String par des 0 a droite pour
   que sa longueur soit un multiple de 8
procedure Complement_Octet(Dans_Fichier : in out
    Unbounded_String);

-- Ecrit un Unbounded_String composé de n octets dans un
   fichier
procedure Ecrire_Binaire(Dans_Fichier : in
    Unbounded_String; Fichier_Comprime : in
    Stream_Access);

procedure Taille_Fichier (File_Name : in out
    Unbounded_String; Taille : out Integer);

```

4 Présentation des principaux choix effectués

L'utilisation d'une structure en arbre de binaire pour l'arbre de huffman semble évidente, pour le reste des structures, on a utilisé des structures en listes chaînées car il s'agit du type de structure qui pose le moins de contraintes et que l'on maîtrise le mieux.

5 L'algorithme Compresser

L'algorithme Compresser doit à partir d'un fichier .txt (fichier à compresser), créer un nouveau fichier avec cette fois ci l'extension .hff qui contient le fichier compressé

Nous avons défini, par soucis de robustesse, des exceptions qui vérifient que le fichier à la bonne extension à savoir .txt et qui lève l'exception `Wrong_Extension_Error` sinon. Ainsi que l'exception `No_Argument_Error` qui est levée si l'utilisateur ne spécifie pas de fichier dans l'invite de commande

L'algorithme fonctionne de la manière suivante :

- A partir du fichier .txt on récupère une liste de taille 256 des fréquences de chacuns des caractères présent dans le fichier à l'aide de la procédure `Creer_Tableau_Frequence_Symbole`. Puis on ajoute à l'indice 256 la valeur -1 qui correspond au `\$` de fin de fichier.
- A partir de la liste des fréquences, on construit les feuilles de arbres de Huffman dans une liste chaînée d'abres binaires, avec l'aide de la

procédure `Enregistrer_Croissant` qui enregistre un arbre de manière croissante en fréquence dans la liste chaînées d'arbre.

- A partir de la liste d'arbres on construit l'arbre de Huffman de manière récursive avec l'aide de la procédure `Huffman`.
- On fait ensuite le parcours infixe de l'arbre avec la procédure `Infixe`
- A partir de l'arbre de Huffman on extrait la table d'encodage à l'aide de la procédure `Generer_Table_Encodage`
- En se basant sur la table d'encodage, on génère la listes des symboles en appliquant le traitement demandé sur le symbole `\$` avec la procédure itérative `Generer_Liste_Symboles`.
- On encode ensuite le texte du fichier d'origine via la table d'encodage avec les procédures `Codage_Texte_Huffman` et `Trouver_Codage`.
- Finalement, on écrit le fichier binaire `.hff` avec la procédure `Ecrire_Binaire`.

6 L'algorithme Décompresser

L'algorithme Décompresser doit à partir d'un fichier `.hff` (fichier compressé), créer un nouveau fichier avec cette fois ci l'extension `.hff.d` qui contient le texte d'origine du fichier `.txt`.

Nous avons défini, par soucis de robustesse, des exceptions qui vérifient que le fichier à la bonne extension à savoir `.hff` et qui lève l'exception `Wrong_Extension_Error` sinon. Ainsi que l'exception `No_Argument_Error` qui est levée si l'utilisateur ne spécifie pas de fichier dans l'invite de commande.

L'algorithme fonctionne de la manière suivante :

- A partir du fichier compressé, on extrait grâce à la procédure `Recuperer_Symboles`, l'ensemble des symboles présents dans le fichier compressé.
- Puis la procédure `Recuperer_Code_Infixe` permet à partir du fichier compressé et de la liste des symboles obtenus précédemment d'extraire le code infixe.
- Une fois la liste des symboles et le code infixe obtenus, on initialise notre arbre d'Huffman, puis la procédure `Recuperer_Arbre`, qui est une procédure récursive, nous permet d'obtenir la structure de l'arbre d'Huffman. Elle fonctionne de la manière suivante :
Si on lit un '0' dans le code infixe alors on créer un nouveau noeud qu'on initialise, on incrémente notre position dans le code infixe de 1, puis on appel la procédure sur le sous arbre gauche et le sous arbre droit à cette

nouvelle position dans le code.

Si on lit un '1' et donc, un caractère, on crée un nouveau noeud dans lequel on écrit le caractère lu à la position dans laquelle on est dans la liste des symboles. Puis on incrémente de 1 notre position dans le code infixe et dans la liste des symboles.

On s'arrête quand on a fini de lire le code infixe.

- Nous avons maintenant tous les outils nécessaires pour décompresser le fichier ce qui est censé être fait par la procédure de compression que nous n'avons pas réussi à faire fonctionner malgré tout notre acharnement dessus.

Cette procédure lit les caractères dans le fichier compressé par octet. On lit l'arbre d'Huffman selon le parcours infixe et quand on est sur une feuille on écrit le caractère correspondant dans le fichier décompressé.

- Enfin, la gestion des options *bavard* et *silencieux* est implémentée de la même manière que pour l'algorithme Compresser.

7 Tests de nos programmes

Étant donné que nous n'avons pas réussi à implémenter totalement l'algorithme Décompresser, l'ensemble des tests effectués porte sur la partie Compresser du projet.

Nous avons dans un premier temps, afin de faire marcher notre algorithme Compresser, créé un fichier *test_lca_abr.adb* qui nous a permis de vérifier que l'on construisait correctement l'arbre de Huffman.

Puis une fois Compresser terminé, nous l'avons testé sur différents types de fichiers :

- *test_long.txt* : test un fichier texte de plusieurs milliers de caractères.
- *test_court.txt* : test un fichier ne contenant qu'un caractère.
- *test_vide.txt* : test un fichier vide.
- *test_retour.txt* : test un fichier ne contenant que des retours à la ligne.
- *test_long.txt -b* : test avec l'option *bavard* précisé
- *test_long.txt -s* : test avec l'option *silencieux* précisé.
- *test_extension.png* : test un fichier ayant la mauvaise extension.
- *** : test le cas où aucun fichier n'est spécifié.

Tout ces tests ce sont montrés concluant, c'est à dire : les fichiers compressés obtenus sont moins volumineux. En les tester avec un algorithme Décompresser fonctionnel, on obtient bien le texte d'origine, donc notre compression est correct. Le programme affiche ce qui est demandé quand l'option *bavard* est activé et n'affiche rien quand c'est l'option *silencieux* qui l'est. Et le programme lève bien les exceptions implémentées quand on est dans les cas correspondants.

8 Difficultés rencontrées et solutions

La première difficultés majeures rencontrées a été de comprendre totalement le principe du codage d'Huffman ainsi que de choisir les types de données que nous allons utiliser pour l'implémenter.

Etant donné que nous venions de finir le TD 8 de PIM et le mini-projet 2 nous nous sommes orientés sur l'utilisation de LCA (listes chaînées associatives) pour ce projet même si ce n'était peut être pas l'option la plus simple. Notamment pour la liste des codes des caractères que nous aurions peut être pu implémenter comme un tableau.

Une autre difficulté rencontrée a été d'apprendre sur le tas l'utilisation de modules que nous n'avions jamais utilisés auparavant tels que : Command_Line ou Streams qui est utile pour gérer des fichiers.

Certaines procédures telle que *Recuperer_Arbre* du programme Decompresser nous a pris plusieurs jours à implémenter.

9 Organisation de l'équipe

Quoi ?		Qui ?			
Modules	Sous-programmes	Spécification	Codage	Test	Relecture
LCA_ABR	Init	Mathis	Mathis	Raphael	Raphael
	Abr_Vide	Mathis	Mathis	Raphael	Raphael
	Associer	Raphael	Raphael	Mathis	Mathis
	Initialiser_LCA_ABR	Mathis	Mathis	Mathis	Raphael
	Est_Vide	Mathis	Mathis	Raphael	Raphael
	Taille	Mathis	Mathis	Raphael	Raphael
	Enregistrement_Croissant	Raphael	Raphael	Raphael	Mathis
	Supprimer	Mathis	Mathis	Raphael	Raphael
	Afficher_Liste_Abre	Raphael	Raphael	Raphael	Mathis
	Huffman	Raphael	Raphael	Raphael	Mathis
	Afficher_Bavard_Huffman	Raphael	Raphael	Raphael	Mathis
	Afficher_Bavard_Huffman2	Mathis	Raphael	Mathis	Mathis
	Infixe	Mathis	Mathis	Raphael	Raphael
	Procedures restantes	Raphael	Raphael	Raphael	Mathis
Compresser	Dans LCA_ABR	Raphael	Raphael	Raphael	Mathis
Décompresser	Recuperer_Symboles	Mathis	Mathis	Mathis	Raphael
	Recuperer_Code_Infixe	Mathis	Mathis	Mathis	Raphael
	Recuperer_Arbre	Mathis	Mathis	Mathis	Raphael
	decompression	Mathis	Mathis	Mathis	Raphael

10 Bilan technique

Au cours de ce projet nous avons totalement implémenter un package afin d'implémenter le codage d'Huffman.

Nous avons réussi à coder l'algorithme Compresser en entier et à le rendre robuste.

Pour ce qui est de l'algorithme Decompresser, nous avons réussi à partir du fichier compressé la liste des symboles ainsi que le code infixe. Notre arbre d'huffman reconstruit est aussi correct sauf à la feuille qui contient le symbole ”\\$” que nous n'avons pas réussi à corriger. Aussi notre procédure qui permet d'afficher l'arbre d'Huffman ne fonctionne pas ici bien que nous avons tenter d'en refaire une toute nouvelle. Enfin la procédure *decompression* ne fonctionne pas non plus et nous n'avons pas réussi à la corriger dans les temps malgré les longues journées passées dessus.

Ce projet à encore de nombreuses perspectives d'améliorations : tout d'abord en débbugant notre programme Decompresser.

Mais aussi des perspectives d'évolutions : nous pourrions essayer de rendre nos programmes génériques et donc compresser avec le codage d'Huffman d'autres types de fichiers tels que des images ou encore des sons.

11 Bilans personnels

- Bilan de Mathis : Personnellement, j'ai dès le départ trouvé ce projet très intéressant car la compression de fichier est essentielle de nos jours en informatique. Ce projet avait un aspect très concret notamment avec la gestion de fichiers (notion que nous n'avions pas vraiment abordé avant je trouve), le taux de compression..

La conception a été compliquée, mis à part au début du projet, il n'y a pas vraiment eu de long moment où l'on se consacrait complètement à la conception. C'est plutôt lors de l'implémentation et que nous faisons face à des problèmes que nous essayons de trouver la solution qui nous conviendrait le plus.

L'implémentation a été la partie la plus longue de ce projet, étant donné que mon camarade et moi venons respectueusement PSI et PT, deux filières éloignées de l'informatique, nous avons eu beaucoup de difficultés dans la programmation de ce projet. Que ce soit au niveau de la compréhension des notions utilisées ou juste de la manipulation des variables et types en ADA qui est un langage avec un typage fort. Nous avons passé plusieurs semaines dessus.

Ce rapport dans lequel je me suis plus investi tandis que Raphael s'occupait plutôt des tests et autres ajustement m'a prit quelques heures.

Ce projet m'a permis pour la première fois de faire l'expérience d'un long travail de groupe, très complexe et exigeant en terme de niveau en informatique et temps.

J'ai au travers de ce projet consolider l'ensemble des notions que nous avons abordés pendant le premier semestre en PIM (raffinages, procédures, types, modules, exceptions) hormis la généricité.

- Bilan de Raphael : Ce fût pour ma part une première expérience très intéressante dans la mise en place d'un projet long en informatique.
L'usage de nouveaux outils tels que git pour la gestion de projet m'as permis d'appréhender plus clairement des projets de développement.
De manière plus évidente, ce projet m'as permis de mieux comprendre les notions employées, que ce soit la syntaxe du langage ADA, ou bien les différentes notions tels que le typage fort, la récursivité, les pointeurs ect.
Le développement du module Compresser a été plutôt complexe, j'ai d'abord eu quelques problèmes pour la compréhension du sujet mais avec l'aide de certains de mes camarades j'ai pu appréhender le sujet, mais aussi dans le choix des types et finalement le développement des procédures et fonctions. Mais, cette difficulté m'as permis de découvrir les methodes classiques de débogage.
Si je dois retenir une chose, c'est la satisfaction de lancer un code fonctionnel après avoir passé des heures à le débogger